

Listas, tuplas, conjuntos y diccionarios

```
[ ]  
( )  
{ }  
{k: v}
```

Lógica de Programación · CEIPA

Viernes 2 de octubre de 2026

Agenda de hoy

1

Colecciones

2

Listas y matrices

3

**Tuplas y
conjuntos**

4

Diccionarios

5

Retos y cierre

Las 4 colecciones de Python

[] LISTA

Ordenada
Modificable

() TUPLA

Ordenada
Inmodificable

{ } CONJUNTO

Sin orden
Sin repetidos

{k: v} DICcionario

Busca por clave

Listas: la navaja suiza

- ```
notas = [3.5, 4.2, 2.8]
notas.append(4.9)
notas.insert(0, 5.0)
notas.remove(2.8)
ultima = notas.pop()
print(notas, len(notas))
print(sum(notas) / len(notas))
```

**Una estructura,  
muchos movimientos**

Agregar · insertar · quitar · medir ·  
sumar

# Cuidado: asignar no es copiar

---



- ```
a = [1, 2, 3]
b = a
b.append(4)
print(a) # [1, 2, 3, 4]
# ¡a también cambió!
c = a.copy() # copia real
```


Comprensión de listas

Ciclo tradicional

```
cuadrados = []  
for n in range(1, 6):  
    cuadrados.append(n ** 2)
```

- ```
cuadrados = [n ** 2 for n in range(1, 6)]

pares = [n for n in range(20) if n % 2 == 0]

aprobados = [nota for nota in notas if nota >= 3.0]
```

# Matrices: listas de listas

---

|   |   |   |
|---|---|---|
| X | O |   |
|   | X | O |
| O |   | X |

```
•
tablero = [["X", "O", " "],
 [" ", "X", "O"],
 ["O", " ", "X"]]

print(tablero[1][2]) # 0
```

# Tuplas y conjuntos

## ( ) TUPLAS

punto = (4, 7)

x, y = punto

Ideales para coordenadas  
y datos fijos.

## { } CONJUNTOS

$a \& b$  intersección

$a | b$  unión

$a - b$  diferencia

set(lista) elimina repetidos.



## RETO EN CLASE 1

# Temperaturas de la semana

Guarda 7 temperaturas. Muestra máxima, mínima, promedio y los días por encima del promedio.

EXTRA · elimina duplicados conservando el orden



**20 min**

# Diccionarios: datos con nombre

## FICHA • ESTUDIANTE

|          |          |
|----------|----------|
| NOMBRE   | Laura    |
| CARRERA  | Robótica |
| SEMESTRE | 2 → 3    |
| PROMEDIO | 4.1      |

```
estudiante = {
 "nombre": "Laura",
 "carrera": "Robótica",
 "semestre": 2
}
print(estudiante["nombre"])
print(estudiante.get("correo", "sin correo"))
estudiante["semestre"] = 3
estudiante["promedio"] = 4.1
```

# Recorrer y métodos principales

---

- ```
for clave, valor in estudiante.items():  
    print(f"{clave}: {valor}")  
  
print(list(estudiante.keys()))  
estudiante.pop("promedio")  
print("nombre" in estudiante)  # True
```

Patrones clásicos: contar y agrupar



CONTEO

```
conteo = {}  
for voto in votos:  
    conteo[voto] = conteo.get(voto, 0) + 1
```



AGRUPAR

```
por_carrera = {}  
for nombre, carrera in estudiantes:  
    por_carrera.setdefault(carrera,  
        []).append(nombre)
```

Diccionarios anidados y JSON

API → JSON

```
curso
└─ estudiantes [ ]
   ├── Ana · 4.5
   └─ Luis · 3.2
```

```
curso = {
    "nombre": "Lógica",
    "estudiantes": [
        {"nombre": "Ana", "nota": 4.5},
        {"nombre": "Luis", "nota": 3.2}
    ]
}
print(curso["estudiantes"][0]["nota"]) # 4.5
```


Caso real: inventario de una tienda

SKU	PRODUCTO	PRECIO	STOCK
A01	Arroz 1 kg	\$4.800	40
B07	Aceite 1 L	\$12.500	3
C15	Café 500 g	\$18.900	0

```
•
por_reponer = [p["nombre"]
    for p in inventario.values()
    if p["stock"] < 5]

valor_total = sum(p["precio"] * p["stock"]
    for p in inventario.values())
```

RETO EN CLASE 2

Palabras más frecuentes

Cuenta cuántas veces aparece cada palabra en un texto y muestra las dos más frecuentes.



15 min

RETO EN CLASE 3

Agenda de contactos

Diccionario de diccionarios + menú para agregar, buscar sin importar mayúsculas, listar y eliminar.



20 min

CIERRE · DÍA 3

Elegir la colección correcta cambia todo.

LISTAS · TUPLAS · CONJUNTOS · DICCIONARIOS

TAREA

Construye un mini inventario con consulta, actualización de stock y reporte de productos por reponer.